Run full Terminal-Bench agentic RL on AWS

A hybrid solution: AgentCore microVMs, m5 EC2 instances and g6 GPU Instances, plus KernelBench

demo_hybrid_terminalbench
slime (Qwen3-4B, fully-async GRPO) on a g6e.48xlarge trainer

Data as of 4 October 2026 (us-west-2)

Key conclusions: every task gets a sandbox; the reward is what is left

Table 1 **Four questions, four answers**

Question	Answer	Evidence	Slides
1. Does it cover the corpus?	**Yes**	**193** usable Terminal-Bench tasks (163 **microVM** + 30 **m5**) vs 165 on microVMs alone, plus **50 KernelBench** tasks on **g6**	slides 9–13
2. Is it reliable?	**Yes**	**4 infra errors in ∼1,900 episodes** over 5 smoke runs, on all three substrates; 0 dirty warm-m5 resets	slide 18
3. Is the sandbox off the critical path?	**Mostly**	pre-open hides opens of 2 s (**microVM**), 68 s (**m5**) and 135 s (**g6**); median start wait 0 / 29 / 28 s	slides 14–16
4. Does the policy learn?	**Not yet**	Terminal-Bench eval 1–4 of 62; 0 of 302 m5 episodes solved; KernelBench needed a speedup gate to give a gradient	slides 19–20

- **One RL code path:** the agent loop, grader, reward, pre-open and metrics do not know which substrate they run on.
- **Routing is data:** each task's substrate comes from its measured build, size and oracle result.
- **One session per sandbox:** a fresh microVM, a warm m5 reused only after a verified reset, a fresh g6 Instance.
- **Runs end to end from 6 scripts**, build to clean-up (slide 22).

Source: Smoke runs 2–6 and `train48`, 4 Oct 2026; `terminalbench/tasks/oracle.csv`; checked 4 Oct 2026.

How to read this deck

Table 2 **Six parts and an appendix; pick the path that fits what you need**

	Part	What you get from it	Starts at
1	**Concepts**	agentic RL in one picture, and the words used in the rest of the deck	slide 4
2	**The problem**	the two workloads, and why one sandbox product cannot run them all	slide 6
3	**The hybrid solution**	architecture, how a task picks its substrate, an episode, sessions, pre-open, security	slide 11
4	**Results**	reliability, reward design, learning	slide 18
5	**Hands-on guide**	prerequisites, the six steps, a healthy run, troubleshooting, your own tasks, cost, IAM, quotas	slide 21
6	**Conclusion**	what the demo shows and what is next	slide 31
A	**Appendix**	how to set every knob, m5 hardening, lessons, references	slide 33

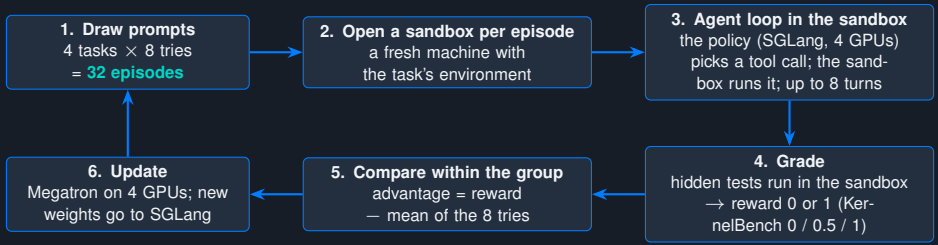
Deciding? Read slide 2 and slide 31.

Designing? Read parts 2–3.

Building? Start at slide 21; tune with the appendix.

Agentic RL in one picture: each step runs 32 agents on 32 machines

Figure 1 One training step (fully-async GRPO in slime)



- **Why sandboxes matter:** every step needs 32 isolated machines in the right state, opened, used for 20–70 s and thrown away.
- **Why the reward matters:** a group where all 8 tries score the same (all 0 or all 1) teaches nothing.
- **Fully-async:** rollout and training overlap, so sandboxes for the next steps are already open while the current step trains.
- **The agent runs inside the sandbox** and calls the trainer's model server for each turn.

Source: `run.sh` (RB=4, NS=8, 4 train + 4 rollout GPUs), `agentcore_slime_demo/terminalbench_pool.py`; checked 5 Oct 2026.

Vocabulary used in this deck

Table 3 **Terms, in the order you meet them**

Term	Meaning here
Episode	one attempt by the agent at one task, from opening a sandbox to the reward
Group	the 8 episodes of the same task in one step; GRPO compares them with each other
Substrate	where a sandbox runs: an **microVM**, an **m5** EC2 instance, or a **g6** GPU Instance
AgentCore Runtime	a registered container image (one per task here); each **session** on it is its own microVM or Instance
Capacity provider	the EC2 fleet AgentCore manages for Instance Runtimes; here g6.xlarge, in AgentCore's own account
Route	the task → substrate address: `arn:...`, `docker://...` or `gpu:arn:...`
Oracle check	before training: the reference solution must pass the hidden tests and the untouched task must fail
Pre-open	opening the next batches' sandboxes ahead of time, so episodes do not wait for a boot
Warm pool	m5 instances kept between episodes and reused only after a verified clean reset
Heartbeat / dead-man switch	a ping that keeps idle microVM sessions alive / an m5 that terminates itself after 30 min without the trainer

Source: This deck.

Terminal-Bench: 241 real terminal tasks, each graded by its own hidden tests

- **What it is:** a benchmark of tasks an agent solves **through a shell**: fix a broken system, configure a service, process data, attack a vulnerable app (laude-institute/terminal-bench).
- **Each task ships:** an instruction, a **Dockerfile** (its environment), a `docker-compose.yaml`, and **hidden pytest tests** plus a reference solution.
- **The reward** (RLVR, no LLM judge): after the agent stops, the tests are copied in and run; reward 1 if they all pass.
- **The agent:** Qwen3-4B with `bash`, `write_file` and `read_file`, up to 8 turns, running **inside** the task's sandbox and calling the trainer for each turn.

Table 4 Corpus

Tasks	n
single container (`client` only)	229
multi-container compose (client + sidecars)	12
total	**241**

In plain English: every task is a small broken computer plus a hidden exam; the agent fixes it with shell commands and the exam scores it.

Source: terminalbench/tasks/manifest.json (fetch_terminalbench.py, laude-institute/terminal-bench main), checked 4 Oct 2026.

KernelBench: write a faster CUDA kernel, graded on a real GPU

- **The task:** given a reference PyTorch `Model`, write `ModelNew` in `solution.py`: same outputs (atol/rtol 10^{-2}), on CUDA, ideally faster (custom kernels, fusion, better ops).
- **Rows:** 40 level-1 tasks (single ops) for training, 10 level-2 tasks (fused ops) held out. They are **mixed into the same batch** as Terminal-Bench rows.
- **The sandbox** needs an NVIDIA GPU: no Terminal-Bench task does, so this is the workload that adds a third substrate.
- **The checker** (hidden, random name, nonce'd `PASSED` line) builds `ModelNew` next to `Model`, checks shape, values and device, and prints `speedup=`.

Table 5 **Reward (`TB_KB_SPEEDUP_MIN` = 1.10)**

Outcome	Reward
correct and $\geq 1.10\times$ faster	**1.0**
correct, not faster	0.5
wrong, crashed or not on CUDA	0

In plain English: the agent must beat PyTorch at its own job, on a real GPU, and a hidden test measures by how much.

Why the reward has a speedup gate: slide 19.

Source: `kernelbench/`, `data/kernelbench_{train,eval}.jsonl`, demo README (KernelBench on the g6 tier); checked 4 Oct 2026.

Three tasks show why each needs its own environment and substrate

Table 6 **Example tasks**

Task	Instruction (abridged)	Environment → substrate
jq-data-processing easy, data processing	"Use jq to transform `/app/users.json`: keep active users, reformat the ISO-8601 last_login dates, write `/app/active_users_report.json`."	One container, small image (jq + data file). **AgentCore microVM**
postgres-csv-clean medium, file operations	"Connect to the Postgres DB from the compose config, import `customers.csv`, clean the data, export a cleaned CSV."	**Two containers**: `client` + a `postgres:16` sidecar on a private network. **m5 EC2** (compose project)
mteb-eval medium, data science	"Evaluate an embedding model from Hugging Face on STS12 with `mteb` so that cosine_spearman $>$ 0.7; store the model in the default cache."	One container, but a **6.7 GB** image (torch + models): over the 2 GB microVM cap. **m5 EC2**

In plain English: the environment decides the substrate: small and single-container goes to a microVM; several containers, x86-only or a big image goes to an m5.

Source: `terminalbench/tasks/manifest.json`, `images.csv`, `route_by_image.json`; checked 4 Oct 2026.

Five things a microVM cannot do, and where those tasks go instead

Table 7 What blocks an AgentCore microVM

Blocker	Tasks	Goes to
Several containers on a private network (compose)	12	m5
Image over the 2 GB cap (`mteb-*`, `png-generation`)	4	m5
No arm64 build (x86-only binaries, dead downloads)	7	m5 if it builds
Reference solution fails on the microVM (e.g. needs capabilities, root or x86)	53	m5 retry
Needs an NVIDIA GPU (KernelBench)	50	g6

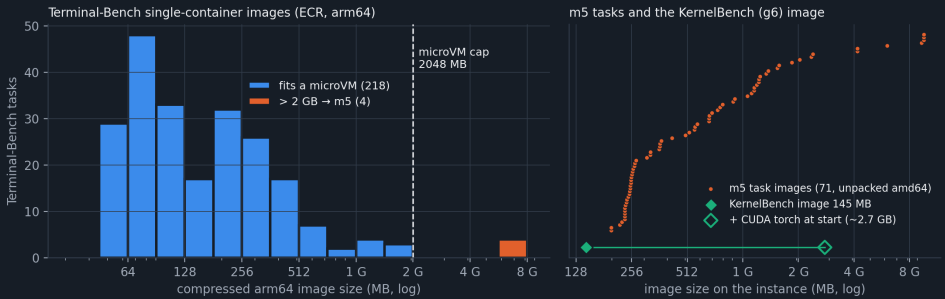
- **AgentCore containers drop all Linux capabilities** and have no Docker daemon, so compose and root-level tasks cannot run inside them.
- **microVMs are arm64 only** and capped at 2 GB per image [Q1].
- **GPUs exist only on AgentCore Instances** (capacity providers), not on microVMs.
- **The m5 is plain EC2 with Docker**: the way Terminal-Bench itself runs upstream.

In plain English: one microVM product covers about two thirds of Terminal-Bench well; the rest needs a whole VM with Docker, and KernelBench needs a GPU.

Source: `images.csv`, `images_docker.csv` (`reason`), `manifest.json`; `docs/terminalbench-unsupported-tasks.md`; checked 4 Oct 2026.

Image sizes: most tasks fit a microVM, the tail and the GPU task do not

Figure 2 **Task image sizes against the 2 GB microVM image cap**



- **arm64 microVM images:** p50 **127 MB**, p95 620 MB, largest that fits 2.0 GB.
- **4 tasks are 6.5–6.7 GB** (`mteb-*`, `png-generation`): no microVM can take them.
- **m5 images** (unpacked, amd64): p50 525 MB, up to 9.8 GB; the m5 has a 40 GB disk.
- **KernelBench:** the 2 GB cap also applies to capacity-provider Runtimes, so the image is 145 MB and installs CUDA torch (∼2.7 GB) at container start (∼90 s).

Source: `images.csv` (ECR compressed size), `images_docker.csv` (`docker image inspect`); `make_figures.py`; checked 4 Oct 2026.

The demo: use case covered, why this demo, design considerations

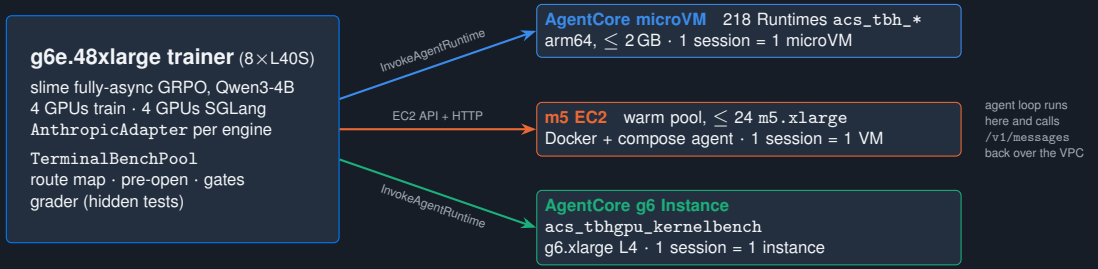
Table 8 **The demo in three answers**

Use case covered	Agentic RL on benchmarks whose environments **do not all fit one sandbox product**: small single containers, multi-container compose projects, x86-only or multi-GB images, root-level tasks, and GPU kernels.
Why this demo	It trains on **the whole usable corpus** (193 Terminal-Bench + 50 KernelBench) instead of the 165 tasks microVMs alone can validate, and shows the sandbox tier can be **heterogeneous without touching the RL side**. `eval/terminalbench` stays the same held-out set as `demo_cpu_terminalbench`.
Design considerations	**One protocol, three transports**: every image is the task's Dockerfile + the same wrapper (`/opt/acs/py` + stdlib `acs_http_server.py`); AgentCore invokes it on a microVM or g6, the trainer POSTs to it on an m5 · **routing is data** (slide 13) · **one session per sandbox** (slide 15) · **per-episode compose rewrite**: fixed names, host ports and `./` mounts made unique; privileged, host network, `docker.sock` refused · **hidden tests stay hidden**: task dirs reach the m5 without `tests/` or the solution · **microVM images reused** byte-for-byte from `demo_cpu_terminalbench`'s ECR repos.
Rejected	Compose on AgentCore (no capabilities, no daemon) · flattening sidecars into one session (changes the task) · the g6e trainer as Docker host (it trains) · ECS/EKS (more moving parts than EC2 for 30 tasks) · g5 for KernelBench (works, but g6 is cheaper: $0.80 vs $1.01/h).

Source: `code/demos/demo_hybrid_terminalbench/README.md`; `docs/terminalbench-hybrid-demo-plan.md`, `docs/docker-compose-on-aws.md`; checked 4 Oct 2026.

One trainer, one router, three sandbox substrates

Figure 3 **Components: the trainer routes each row; the agent loop runs inside the sandbox and dials back**



microVM: 163 tasks. Managed by AgentCore; an idle session is wiped after 15 min, so a heartbeat keeps pre-opened ones alive.

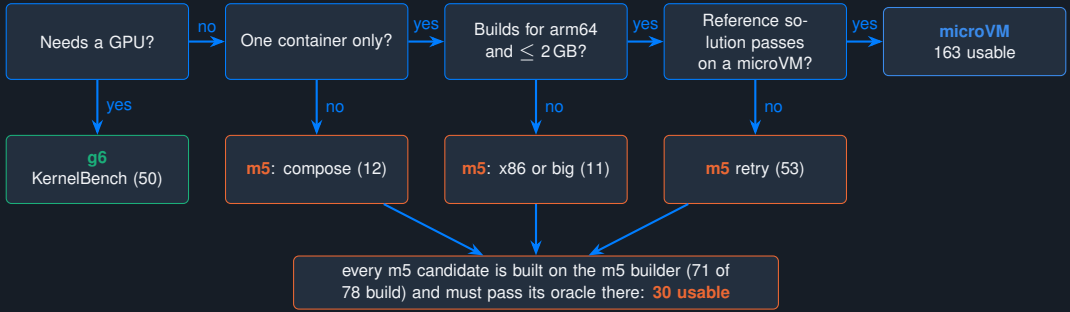
m5: 30 tasks. **Our code** launches and terminates them (boto3, the g6e's role), from a baked AMI; images from ECR.

g6: KernelBench. In an AgentCore-managed account: **not in our EC2 console**; one CloudWatch log stream per session.

Source: `agentcore_slime_demo/{terminalbench_pool, docker_compose_sandbox, ec2_episode_sandbox}.py`, `image_routing.py`, `run.sh`; checked 4 Oct 2026.

Which substrate does a task get? Four questions, answered from data

Figure 4 The routing decision, with the number of Terminal-Bench tasks on each branch



- **193 of 241 Terminal-Bench tasks are usable** (163 + 30) vs 165 with microVMs alone. Every answer is measured: `manifest.json` (containers), `images.csv` (build, size), `oracle.csv` (oracle). `image_routing.py` refuses a task with no route.
- **Not usable (48):** 5 build nowhere (e.g. `fix-git`: its GitHub repo was deleted); the rest fail their oracle on every substrate, incl. the 4 big `mteb-*/png-generation` images.

Source: `terminalbench/tasks/{manifest.json, images.csv, images_docker.csv, oracle.csv, route_by_image.json}`; checked 5 Oct 2026.

One episode is the same five steps on every substrate

Table 9 What `TerminalBenchPool` does per episode, by substrate

Step	microVM	m5 EC2 (warm)	g6 Instance
1. Route	`arn:.../acs_tbh_<task>`	`docker://<ip>:18300/<task>`	`gpu:arn:.../acs_tbhgpu_kernelbench`
2. Open (ahead)	new `runtimeSessionId`; first invoke boots the microVM	idle VM (prefer one holding the images) or `run_instances`; ECR pull; `compose up`	new session id; first invoke launches a g6, then torch install
3. Agent loop	mini_claude (`bash` / `write_file` / `read_file`), up to 8 turns × 1024 tokens, **inside the sandbox**, calling the trainer's adapter		
4. Grade	inject `tests/`, run pytest	same, in the client	hidden checker: correctness + `speedup=`
5. Close	`StopRuntimeSession`	`compose down -v` + `reset_check`; clean → pool, dirty → terminate	`StopRuntimeSession` (instance released)
Open cost (smoke 6, p50)	**2 s**	**68 s** (launch or reuse, pull, `compose up`)	**135 s** (boot + ∼90 s torch)
Episode p50	24 s (agent 12 s, grade 9 s)	69 s (agent 14 s, grade 7 s)	30 s (first command → stop; p90 41 s)

In plain English: the RL code never asks which substrate it is on; only the open cost differs (2 s to over 2 min).

Source: `logs/smoke6-hybrid_*.log` (`[demo] agent_ms, grade_ms, episode_ms, open_ms`); m5 per-episode probe; checked 4 Oct 2026.

Sessions and instances: one session per instance, three lifecycles

Table 10 Who owns each instance, and how it ends

	microVM	m5	g6
Managed by	AgentCore	**our trainer**	AgentCore
Reused?	no, fresh	yes, after a verified reset	no, fresh
Isolation	own kernel	own VM per session	own instance
Visible in	CloudWatch	our EC2 console	CloudWatch only
Ends by	stop / 15-min idle	5 ways (right)	stop / 600 s idle
Peak live (`train48`)	88	24	36

Table 11 Warm m5 pool in smoke 6

instances launched / episodes on a reused one	24 / 10
pool peak (cap `TB_M5_MAX_INSTANCES`)	24 (24)
dirty resets / retired	**0 / 0**

- **Verified reset** after every m5 episode: no containers, volumes or extra networks, empty episode dir, $\geq$ 8 GB disk. Dirty or unhealthy $\rightarrow$ terminated, never reused. No periodic recycle.
- **m5 termination, 5 ways:** trainer `atexit`; `5_run.sh gc-episodes`; next pool start; `6_cleanup.sh`; a **dead-man switch** (no trainer call for 30 min $\rightarrow$ the VM shuts itself down = terminate).
- **g6:** a launch per session, so a pre-open must start $\sim$2.5 min early (slide 16).
- Isolation and hardening details: slide 36.

Source: `agentcore_slime_demo/ec2_episode_sandbox.py` (`WarmPool`); `logs/smoke6-hybrid_*`, `logs/train48-hybrid_*` (`[sbx-op]`, `warm_reused`); checked 4 Oct 2026.

Pre-open hides cold starts from 2 s to 2.5 min behind the running batch

Figure 5 **Median wait for the sandbox at episode start: measured with pre-open vs the open time it hides**

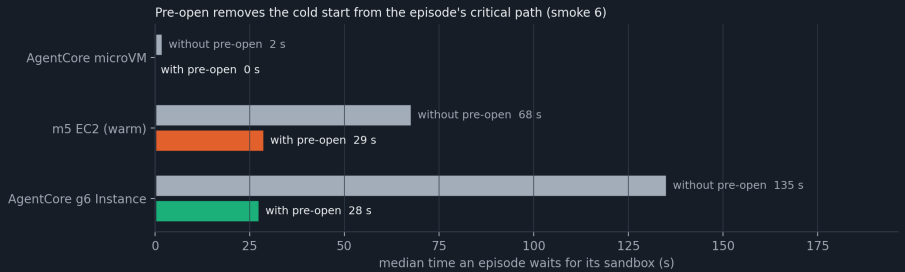


Table 12 **Smoke 6, by substrate**

	n	no wait	wait p90	saved
microVM (2 batches ahead)	177	121	5.6 s	4 min
m5 (5 batches ahead)	34	16	61 s	23 min
g6 (5 batches ahead)	52	22	100 s	**80 min**

Table 13 **g6 session cap vs start wait**

Run	cap	wait p50	no wait
smoke 4	8	240 s	0 / 76
smoke 5	16	82 s	5 / 36
smoke 6	32	**28 s**	22 / 52

A cap of 8 equals one group, so nothing could open ahead.

Source: `logs/smoke{4mix,5,6}-hybrid_*.log` (`[demo] open_wait_ms, open_ms`); "saved" = $\sum$ max(0, open − wait); `make_figures.py`; checked 4 Oct 2026.

Security and network: private by design, no public ports

Table 14 Where each component runs and who can reach it

Component	Placement	Inbound allowed	Outbound
microVM / g6 sessions	AgentCore **VPC mode**, our private `acs-egress-<az>` subnets	none: reached only through the IAM-signed AgentCore API	existing NAT (pip at grading); AWS via private endpoints
m5 episodes + builder	private egress subnet, **no public IP**	tcp 18300 (agent) and 32768–60999 (episode client) **from the trainer's SG only**; no SSH rule	existing NAT; IMDS blocked from containers
Trainer (g6e)	our VPC	tcp 18100–18163 (model adapters) **from the Runtime and m5 SGs only**	–

- **Private endpoints** (created by step 4): `ecr.api`, `ecr.dkr`, `logs`, `sts`, `bedrock-agentcore`, and an S3 gateway for ECR layers.
- **No NAT gateway is created**; an existing one is required for tasks whose tests install packages. `RUNTIME_EGRESS=none` = no internet at all.
- **No credentials worth stealing** in a sandbox: the m5 role can only pull its own images (slide 28).
- **The m5 agent API** needs a per-instance secret; it is plain HTTP inside the VPC, limited by the SG-to-SG rule.
- **Ray dashboard** (8265) listens on the trainer but no SG rule exposes it.
- **Dev-account note:** our trainer host has a public IP and **SSH open to 0.0.0.0/0**. The solution does not need it: restrict it to your IP or use SSM Session Manager.

Source: `deploy/agentcore_deploy.py` (`RUNTIME_NETWORK=VPC`, `vpc_network`, endpoints), `deploy/m5_host.py` (SG rules, `AssociatePublicIpAddress=False`); `aws ec2 describe-security-groups`; checked 5 Oct 2026.

Reliability: 4 infra errors in about 1,900 episodes, on all three substrates

Table 15 **Smoke runs on 4 Oct 2026: episodes (train + eval) / infra errors**

Run	microVM	m5	g6	Run result and what changed after it
smoke 2	401 / 1	88 / 0	–	final checkpoint save hung (`pidfd_getfd` EPERM) → allocator setting made opt-in
smoke 3	382 / **0**	88 / **0**	–	**passed**, 53 GB checkpoint; m5 per-episode → warm pool next
smoke 4	322 / 1	58 / 1	75 / 0	**passed**; KernelBench added; g6 cap 8 too small
smoke 5	157 / 0	34 / 0	36 / 0	**passed**; speedup-gated reward, g6 cap 16
smoke 6	177 / 1	34 / 0	52 / 0	**passed**; g6 cap 32, 5 batches ahead
total	1,439 / 3	302 / 1	163 / 0	**1,904 episodes, 4 infra errors (0.2 %)**

- **Every run since smoke 3 finished** (`Job ... succeeded`); smokes 3–4 saved a 53 GB checkpoint.
- **0 dirty warm-m5 resets**; all m5s terminated at exit.
- `train48` **run 1:** 1 infra error in 1,876 episodes, then an eval deadlock (slide 20).
- Bugs found on the way and their fixes: slide 37.

Source: `logs/smoke{2,3,4mix,5,6}-hybrid_*.log` (`[demo] infra_error=`); `make_figures.py` → `data.json` (runs); checked 4 Oct 2026.

Reward design: KernelBench needed a speedup gate to give a gradient

Table 16 Reward per run (TB: solved / episodes; KB: count per reward)

Run	TB microVM	TB m5	KernelBench
smoke 3	16 / 382	0 / 88	–
smoke 4	13 / 322	0 / 58	1: 48, 0: 27
smoke 5	6 / 157	0 / 34	0.5: 26, 0: 10
smoke 6	4 / 177	0 / 34	1: 8, 0.5: 34, 0: 10
eval (s6)	1 / 62	0 / 10	mean 0.45

- **GRPO learns only from groups with mixed rewards.** All-0 or all-1 groups give no gradient.
- **KernelBench, correctness only** (smoke 4): copying `Model` onto CUDA passed, so 64 % scored 1.
- **Speedup gate** (smoke 5–6): 1 only if $\geq 1.10\times$ faster. Copies measure 1.00–1.03×, so the bar is above noise. Smoke 6 had all three reward levels; best kernel 1.38×.
- **Terminal-Bench is all-or-nothing**: a 4B model solves 2–4 % on microVMs and **0 %** of the harder m5 tail, so most groups carry no gradient.

Source: `logs/smoke{3,4mix,5,6}-hybrid_*.{log,eval.csv}` (`solved=`, `reward=`, `speedup=`); checked 4 Oct 2026.

Learning: the 48-step training run (re-run in progress)

Table 17 **Run 1 (4 Oct 15:24 UTC): 20 steps, then an eval deadlock**

steps done	20 of 48
mean step time	130 s
episodes (microVM / m5 / g6)	1,397 / 151 / 328
solved (microVM / m5 / g6)	2.0 % / 0 % / 59 %
infra errors	1
eval at step 10 (TB / m5 / KB)	1/62 / 0/10 / 0.425
peak live sandboxes (microVM / m5 / g6)	88 / 24 / 36

Train reward per step 0–19: 0, 0, .25, .25, 0, .13, .11, 0, .02, 0, .08, 0, 0, .22, 0, .13, 0, .13, 0, –

- **What happened:** at the step-20 eval, all 24 warm m5 slots were still held by sessions the paused training loop had pre-opened. That loop was wedged and never released them, so the m5 eval waited for an instance forever (8 h, GPUs idle).
- **No cost leak:** the dead-man switch terminated every idle m5 after 30 min.
- **Fix:** the warm pool charges each instance to the event loop holding it; a superseded loop's instances are reclaimed and terminated (slide 37).
- **Run 2** started 5 Oct 00:56 UTC with the fix; results go here when it ends.

Source: `logs/train48-hybrid_20261004_152433.{log,csv,eval.csv}` (`metrics.py` by substrate); py-spy dump of the rollout process; checked 5 Oct 2026.

Before you start: what you need, and how to check it

Table 18 **Prerequisites**

Item	What	Check
Account and Region	us-west-2 (AgentCore Runtime and capacity providers)	`aws sts get-caller-identity`
Trainer host	g6e.48xlarge (8 × L40S); disk for 53 GB per checkpoint	`nvidia-smi`; `df -h`
Trainer container	`agentcore_p4` (slime image, Qwen3-4B), repo mounted at `/work`	`docker exec agentcore_p4 nvidia-smi`
Image builds	Docker buildx with arm64 emulation on the host (microVM images are arm64)	`docker buildx ls`
Network	default VPC; an existing NAT gateway (none is created)	`aws ec2 describe-nat-gateways`
IAM	a trainer role with the actions on slide 29	–
Quotas	≥ 220 free Runtimes, ≥ 100 standard vCPU (slide 30)	`aws service-quotas ...`
Python on the host	`boto3`	`python3 -c "import boto3"`

- **Run the step scripts on the host**, not in the container: they `docker exec` into `agentcore_p4` when they need it. Running `run.sh` on the host fails on `/root/slime`.
- **Dry run first:** `TB_LIMIT=2` on steps 2 and 4 catches IAM, network and image errors in minutes (slide 22).

Source: `deploy/steps_common.sh`, `README.md`; checked 5 Oct 2026.

End to end, part 1: build the images, push to ECR, deploy and validate

From `code/demos/demo_hybrid_terminalbench/`, on the trainer host

```
# optional dry run on 2 tasks first
TB_LIMIT=2 bash 2_build_image.sh
TB_LIMIT=2 bash 4_deploy_runtime.sh

# 1. build every sandbox image, push to ECR
bash 2_build_image.sh
# 2a arm64 images -> ECR acs-tb-<task>
# 2b launch the m5 builder (SGs, Docker)
# 2c m5 tasks built for amd64 on it
# -> ECR acs-tbhdk-<task>-<service>
# 2d bake the per-episode m5 AMI
bash 3_push_ecr.sh # report: what shipped

# 2. register Runtimes + validate oracles
bash 4_deploy_runtime.sh
# 4a 218 microVM Runtimes acs_tbh_<task>
# 4g g6 capacity provider + KernelBench
# 4b-d oracle on every route; microVM
# failures retried on the m5
```

Table 19 What each step produces

`2_build_image`	`manifest.json` (241 tasks), `images.csv`, `images_docker.csv`, ECR repos, AMI, `deploy/m5_host.json`
`3_push_ecr`	a report only (step 2 pushed)
`4_deploy_runtime`	`route_by_image.json` (`arn:` / `docker:///` / `gpu:`), `oracle.csv`

- **Prerequisites:** the `agentcore_p4` container with slime + Qwen3-4B, Docker buildx (arm64), AWS credentials with the trainer role's permissions (slide 28), quotas (slide 30).
- **Idempotent and resumable:** images already in ECR are reused, Runtimes are updated, not duplicated.
- `TB_SUBSTRATES=microvm` or `=docker` runs one half.
- Every step costs real AWS money from 2b on.

Source: `2_build_image.sh`, `3_push_ecr.sh`, `4_deploy_runtime.sh`, `README.md` (Step by step); checked 4 Oct 2026.

End to end, part 2: prepare data, train, evaluate, clean up

```
# 3. rows for every routed + validated task
bash 1_prepare_data.sh
# data/terminalbench_train.jsonl (both)
# data/terminalbench_eval.jsonl (microVM)
# data/terminalbench_eval_docker.jsonl
# data/kernelbench_{train,eval}.jsonl

# 4. train + evaluate (one command)
bash 5_run.sh # 10-step smoke
RUN_TAG=train48 NR=49 EVAL_INTERVAL=10 \
  bash 5_run.sh # 48 steps
# eval every 10 steps + at the end
# -> logs/<tag>.{log,csv,eval.csv,png}

# 5. tear down (ECR images, data, ckpt kept)
bash 6_cleanup.sh # KEEP_M5=1
```

Table 20 What to read after a run

`<tag>.csv`	per step: `rollout/raw_reward`, `perf/step_time`, `perf/sandbox_*`
`<tag>.eval.csv`	`eval/terminalbench`, `eval/terminalbench_docker`, `eval/kernelbench`
`<tag>.demo.csv`	per episode: substrate, solved, open / agent / grade ms
`<tag>.png`	reward and timing curves
`ckpt/`	final checkpoint (53 GB)

- **Evaluation runs inside training** on the held-out sets: 31 microVM tasks, the m5 eval tasks and 10 KernelBench level-2 tasks, 2 samples each.
- `5_run.sh` runs the trainer **inside** `agentcore_p4` (`docker exec`), then sweeps m5s.
- **Timing:** smoke 6 (3 steps + eval) 18 min; `train48` $\approx$ 2.5 h.
- **Never edit** `run.sh` or a step script while it runs: bash reads them as it goes.

Source: `1_prepare_data.sh`, `5_run.sh`, `6_cleanup.sh`, `train.sh`, `metrics.py`; `logs/step5_train48.out`; checked 4 Oct 2026.

What a healthy run looks like, and where to look

Table 21 Signals in order of a run (`logs/<tag>.log` unless noted)

When	You should see	Also in
Start	`[image_routing] 243 rows -> 194 image(s): 163 ... microVM ... + 30 m5 ... + 1 g6`	–
First minutes	`[sbx-op] op=ping ... status=ok` on all three substrates; g6 first ping after ∼2.5 min	CloudWatch: `/aws/bedrock-agentcore/runtimes/<id>-DEFAULT`
Each episode	`[demo] episode_end ... infra_error=none`	`<tag>.demo.csv`
Each step	`rollout/raw_reward`; `perf/step_time` ≈ 130 s	`<tag>.csv`, `<tag>.png`
Every 10 steps	`Eval terminalbench` 62/62, then the m5 and KernelBench evals	`<tag>.eval.csv`
End	`Job '...' succeeded`; checkpoint in `ckpt/`; m5s swept	EC2 console: no `acs-tbh-m5-ep` left

Table 22 Warning signs

- the log is silent for > 15 min, or GPUs sit at 0 %
- `infra_error=` anything but `none`
- `open_wait_ms` growing step after step

Table 23 Quick commands

- `tail -f logs/<tag>.log | grep -E "episode_end|raw_reward"`
- `python3 metrics.py logs/<tag>.log` (by-substrate summary)
- Ray dashboard: `http://<trainer>:8265` (via SSM port forward)

Source: `logs/train48-hybrid_*.log`, `metrics.py`, `5_run.sh`; checked 5 Oct 2026.

Troubleshooting: symptoms we hit, their cause, and the fix

Table 24 Seen while building and running this demo

Symptom	Likely cause	What to do
Runtime `CREATE_FAILED`, no reason, empty log group	VPC session cannot reach a private endpoint	check endpoints are `available`, the S3 gateway route, and that the subnet's AZ has every endpoint
`No CUDA GPUs are available` in the container	a Docker daemon reload dropped the GPU cgroup	`sudo docker restart agentcore_p4`
`run.sh: /root/slime ... Permission denied`	launched on the host	use `bash 5_run.sh` (it runs inside the container)
Episodes wait long at start	pre-open too shallow or a cap too low	raise `TB_PREOPEN_BATCHES_SLOW` or the cap (slide 33)
Log silent, GPUs at 0 %	a wedged eval (the step-20 deadlock, now fixed)	`py-spy dump -pid <RolloutManager>` in the container; look for `warm_reclaim`
Reward 0 on every task	tool calls not parsed (`Failed to parse JSON`)	check the tool parser in `run.sh` and `tool_calls=` in `[demo]` lines
m5 instances left after a crash	the trainer died before its clean-up	`python3 deploy/m5_host.py gc-episodes`; the dead-man switch is the backstop
Strange errors after an edit	bash reads a running script as it goes	never edit `run.sh` or a step script while it runs

Source: Smoke runs 2–6, `train48`; notes in the demo README; checked 5 Oct 2026.

Bring your own tasks: the same pipeline routes, checks and trains them

Table 25 **A task in Terminal-Bench format**

`task.yaml`	instruction, difficulty, test timeout
`Dockerfile`	the environment, with seed files
`docker-compose.yaml`	`client` (+ sidecars for the m5)
`tests/`, `run-tests.sh`	the hidden pytest suite
`solution.sh`	reference solution, for the oracle check

1. Put it in `terminalbench/tasks/<id>/` and add it to `tasks/manifest.json`.
2. `bash 2_build_image.sh`, then `bash 4_deploy_runtime.sh`: it is built, routed and oracle-checked.
3. `bash 1_prepare_data.sh`: only tasks that pass the oracle become rows.

Table 26 **Rules that keep the reward honest**

- The reference solution must **pass** and the untouched task must **fail**.
- Never bake `tests/` or the solution into the image: they are injected only at grading.
- Prefer partial credit where you can (tests passed / total): all-or-nothing gives few gradients.

Table 27 **A different benchmark (the KernelBench pattern)**

- one image with the same wrapper server; a hidden checker that prints a nonce'd result
- rows whose `image` names a route (`arn:`, `docker://` or `gpu:`)
- a reward function that reads the checker's output

Re-running `fetch_terminalbench.py` recreates `tasks/`: keep your tasks elsewhere and copy them in.

Source: `terminalbench/fetch_terminalbench.py`, `deploy_terminalbench.py`, `validate_oracle.py`, `kernelbench/`; checked 5 Oct 2026.

What a run uses, and how to keep the bill down

Table 28 **Resources per hour of training (`train48` run 1: 1 h, 20 steps)**

Resource	Used
Trainer g6e.48xlarge	1 instance-hour (also while idle or hung)
microVM sessions	781 sessions, **27 session-hours**
g6 Instances	224 sessions, **16 instance-hours** (1 per session)
m5 warm pool	98 episodes on $\leq$ 24 m5.xlarge, $\leq$ 24 instance-hours
m5 builder	1 m5.xlarge, only needed to build
ECR	$\sim$49 GB arm64 images + the m5 images
Checkpoints	53 GB each (`ckpt/` holds 210 GB after 4 smokes)

Prices: AgentCore pricing [P1] and EC2 On-Demand pricing [P2] (slide 38).

Table 29 **Cost controls**

- `6_cleanup.sh` after the last run; stop the builder.
- m5s are swept after every run, and terminate themselves after 30 min without the trainer.
- g6 sessions stop at the episode's end; idle g6s go after 600 s.
- `TB_KERNELBENCH=0` drops the g6 tier; `TB_M5_MAX_INSTANCES` caps the m5s.
- Watch for hangs: the trainer bills while stuck.
- Delete old checkpoints.

Source: `logs/train48-hybrid_20261004_152433.{sessions,sandbox_ops}.csv`; `images.csv`; `du -sh ckpt`; checked 5 Oct 2026.

IAM: five roles, each limited to its own job

Table 30 **Roles the solution uses (names with `NAME_PREFIX=acs`)**

Role	Assumed by	Permissions	Created by
Trainer instance role (on the g6e)	EC2 (the trainer host)	builds, deploys and runs everything: see the minimum set on slide 29	you, before step 2
`acs-exec-role`	`bedrock-agentcore` (`aws:SourceAccount` = ours)	ECR pull (`GetAuthorizationToken`, `BatchGetImage`, `GetDownloadUrlForLayer`, `BatchCheckLayerAvailability`); CloudWatch Logs write; `InvokeAgentRuntime`; VPC ENI create/describe/delete	step 4
`acs-operator-role`	`bedrock-agentcore`	AWS managed `BedrockAgentCoreRuntimeInstancesOperatorRolePolicy`: lets AgentCore launch the g6 capacity-provider instances	step 4g
`acs-tbh-m5-builder`	EC2 (the m5 builder)	ECR pull + push + `CreateRepository` on `acs-tbhdk-*` only	step 2b
`acs-tbh-m5-episode`	EC2 (episode m5s)	**ECR pull on** `acs-tbhdk-*` **only**; nothing else	step 2b

- **The sandboxes hold no useful credentials:** the agent runs code an RL policy wrote, so the episode role can only pull its own images, and IMDS is blocked from containers (slide 36).
- **Trust policies** are service-scoped; AgentCore roles add an `aws:SourceAccount` condition.
- **Our dev trainer role is much broader** than needed (e.g. `AmazonEC2FullAccess`, `IAMFullAccess`, `bedrock-agentcore:*`). For a customer, use the minimum set on the next slide.
- The g6 instances run in AgentCore's service account; our account sees only the Runtime and its logs.

Source: `deploy/agentcore_deploy.py` (`exec_role`, `operator_role`), `deploy/m5_host.py` (`_ensure_profile`, `roles`), `ec2_episode_sandbox.py`; `aws iam list-attached-role-policies`; checked 4 Oct 2026.

IAM: the minimum the trainer role needs, by phase

Table 31 Trainer instance role: actions the code calls

Service	Build and deploy (steps 2–4, 6)	Train and evaluate (step 5)
bedrock-agentcore	CreateAgentRuntime, UpdateAgentRuntime, GetAgentRuntime, ListAgentRuntimes, DeleteAgentRuntime; Create/Get/List/DeleteCapacityProvider	InvokeAgentRuntime, StopRuntimeSession
iam	CreateRole, GetRole, UpdateAssumeRolePolicy, PutRolePolicy, AttachRolePolicy, CreateInstanceProfile, AddRoleToInstanceProfile; PassRole on the 4 roles above	PassRole on acs-tbh-m5-episode
ec2	RunInstances (builder, AMI bake), CreateImage, DescribeImages; Create/Delete/DescribeSecurityGroups, Authorize/RevokeSecurityGroupIngress; Describe{Vpcs,Subnets,RouteTables,NatGateways}, VPC endpoints; AssociateIamInstanceProfile; TerminateInstances	RunInstances, CreateTags, DescribeInstances, TerminateInstances (tag acs-tbh-m5-ep)
ecr	GetAuthorizationToken, CreateRepository, DescribeRepositories, DescribeImages, BatchGetImage, layer upload + PutImage (arm64 push from the host)	–
logs, sts	logs:Get/FilterLogEvents (optional), sts:GetCallerIdentity	sts:GetCallerIdentity

- **Scope it down:** Resource = arn:aws:bedrock-agentcore:<region>:<acct>:runtime/acs_tbh*, ECR repository/acs-tb*, IAM role/acs-*; condition ec2:ResourceTag/acs-tbh-m5-ep on TerminateInstances.
- **Training needs only the right-hand column**, so a separate, smaller role can run step 5 once the pool is built.

Source: boto3 calls in deploy/, terminalbench/, agentcore_slime_demo/ (grep of ec2., iam., ecr., bedrock-agentcore); checked 4 Oct 2026. Not yet tested as a standalone policy.

AWS quotas: what the solution uses against each limit

Table 32 **Quotas in us-west-2 (account values from `aws service-quotas`, 4 Oct 2026)**

Quota (code)	Limit	We use	Note
AgentCore			
Runtimes per account [Q1]	1,000	**440**	219 this demo + 218 leftover `acs_tb_*`: delete before adding a 3rd pool
Docker image size, MB (L-0A9E32B3)	2,048	max 2,003	why 4 tasks go to the m5 and torch installs at start
Active session workloads (L-3E5722B2)	5,000	88 + 36 peak	microVM + g6 sessions live at once (`train48`)
New Runtime sessions per s (L-8EE2AEA2)	25	bursts at pre-open	pre-open spreads opens over the batch
Runtime data-plane API rate (L-46ED137C)	1,000/s	low	invoke + ping + stop
Control-plane mutations (L-DD349033)	50/s	step 4 only	creates 219 Runtimes in a loop
Request timeout (L-3ED45A13)	15 min	180 s/exec	`TB_EXEC_TIMEOUT`
Session idle / max lifetime [D2]	15 min / 8 h	pings at 7.5 min	heartbeat keeps pre-opened sessions
EC2 and VPC			
On-Demand Standard vCPU (L-1216C47A)	1,152	100	24 × m5.xlarge (4) + builder (4)
On-Demand G and VT vCPU (L-DB2E81BA)	768	192	the g6e.48xlarge trainer
Network interfaces per Region (L-DF5E4CA3)	5,000	not measured	VPC-mode sessions + m5s; check before scaling up
Rules per security group (L-0EA8095F)	60	few	SG → SG 18100–18163, 18300
gp3 storage, TiB (L-7A658B76)	257	∼1.5	24 × 40 GB + 500 GB builder

Source: `aws service-quotas list-service-quotas --service-code bedrock-agentcore|ec2|vpc|ebs`; peaks from `logs/train48-hybrid_* [sbx-op]`; `list_agent_runtimes`; [Q1], [D2] on slide 38; checked 4 Oct 2026.

Conclusion: every task gets a sandbox it runs on; the reward is what is left

Table 33 What the demo shows

	Conclusion	Evidence	Slides
1	**A hybrid tier covers the corpus**	193 usable Terminal-Bench tasks vs 165 on microVMs alone, plus KernelBench on g6	slides 9–13
2	**The RL side does not change**	one agent loop, grader, pre-open and metrics path for microVM, m5 and g6	slides 12–14
3	**Infra is reliable**	4 infra errors in 1,904 smoke episodes; 0 dirty warm resets	slide 18
4	**Pre-open depth must match boot time**	g6 start wait p50 240 s $\rightarrow$ 28 s with cap 32 and 5 batches ahead	slide 16
5	**The reward limits learning**	TB eval 1–4 of 62; 0 of 302 m5 episodes solved; KernelBench needed a speedup gate	slide 19

Table 34 Next steps

1	Finish the 48-step run	re-run with the deadlock fix, started 5 Oct 00:56 (slide 20)
2	Partial credit from `tests_passed` / `tests_total`	more GRPO groups with a gradient, esp. on m5 tasks
3	Host the CUDA torch wheel in S3	cut the g6 open from $\sim$2.5 min
4	Delete the 218 leftover `acs_tb_*` Runtimes; stop the m5 builder	frees 22 % of the Runtime quota (slide 30); builder is not needed to train

Source: This deck; checked 4 Oct 2026.

Thank you

`code/demos/demo_hybrid_terminalbench`

Appendix: how to configure each setting, m5 hardening, lessons, references

How to configure (1 of 3): pre-open depth and instance caps

Set any setting on the command line; `5_run.sh` passes every `TB_*`, `NR`, `RB`, `NS`, `EVAL_*` into the trainer container:

```
TB_PREOPEN_BATCHES_SLOW=5 TB_GPU_MAX_SESSIONS=32 TB_M5_MAX_INSTANCES=24 RUN_TAG=mytest bash 5_run.sh
```

Table 35 **Settings that decide how much is opened ahead, and how many instances may be live**

Setting	Default	How to choose	Too low / too high
`TB_PREOPEN_BATCHES` (microVM)	2	batches opened ahead $\geq$ open time / batch time; a 2 s open needs 1	0: every episode pays the boot / idle microVMs billed
`TB_PREOPEN_BATCHES_SLOW` (m5, g6)	5	same rule for the slow opens: g6 $\sim$150 s, m5 $\sim$70 s cold	3 (with cap 16): g6 waited 82 s / more idle instances
`TB_GPU_MAX_SESSIONS`	32	$\geq$ running g6 episodes + pre-opened ones: KernelBench prompts in the look-ahead $\times$ `NS` ($\approx 4 \times 8$)	8 (= one group): wait 240 s; 16: gate full 66 % of the run / more live g6s
`TB_M5_MAX_INSTANCES`	24	peak concurrent m5 demand (24 covers $\sim$87 % of moments); 4 vCPU each against the quota	episodes queue for an m5 / idle VMs billed, vCPU quota

Source: `run.sh`, `deploy/steps_common.sh` (`PASS_RE`), README (Knobs); evidence from smoke 4–6 (slide 16); checked 5 Oct 2026.

How to configure (2 of 3): m5 lifecycle and robustness

Table 36 Settings that decide how sandboxes are reused, kept alive and retried

Setting	Default	How to choose	Too low / too high
`TB_M5_MODE`	warm	`warm`: reuse after a verified reset; `per_episode`: new VM each time (+34–51 s); `shared`: oracle checks only	–
`TB_M5_IDLE_SHUTDOWN_S`	1800	longer than the longest normal silence (final save + eval ≈ 10 min)	warm m5s replaced mid-run / orphans bill longer after a crash
`HEARTBEAT`	1	keep on whenever pre-opened sessions can idle past 15 min	sessions silently wiped / negligible cost
`TB_EXEC_TIMEOUT`	180 s	longest legitimate command (builds, installs)	agent cut off / hung commands hold a sandbox
`INVOKE_EXEC_MAX_RETRIES`	1	`exec` is not idempotent: keep at 1	– / 9 retries of a 4-min call stalled an eval

Table 37 Recipes

microVM tasks only, cheapest	`TB_KERNELBENCH=0`, and `TB_SUBSTRATES=microvm` in steps 2 and 4
strongest m5 isolation	`TB_M5_MODE=per_episode` (each episode on a VM terminated after grading)
convergence run	`RUN_TAG=train48 NR=49 EVAL_INTERVAL=10 bash 5_run.sh`

Source: `run.sh`, README (Knobs, Warm m5 pool); checked 5 Oct 2026.

How to configure (3 of 3): run length, batch shape, agent and reward

Table 38 **Training settings**

Setting	Default	How to choose	Watch out for
`NR`	11	train steps + 1 (`train.sh` takes `STEPS`); one epoch = 197 train rows / `RB` 4 $\Rightarrow \leq$ 48 steps	each epoch boundary writes another 53 GB checkpoint
`EVAL_INTERVAL` / `EVAL_NS`	10 / 2	eval every N steps (and at the end); samples per eval prompt	46 eval prompts $\times$ 2 add $\sim$20 min per eval
`RB` $\times$ `NS`	4 $\times$ 8	prompts $\times$ GRPO group size = 32 episodes per step	sandbox demand scales with `RB`$\times$`NS`$\times$look-ahead
`EPISODE_MAX_TURNS` / `_TOKENS`	8 / 1024	raise for harder tasks; most m5 episodes hit 8 turns	longer episodes, larger micro-batches
`TB_KB_SPEEDUP_MIN`	1.10	above the 1.00–1.03$\times$ noise of copying `Model`; 0 = correctness only	0: groups of all 1s, no gradient
`TB_KERNELBENCH` / `TB_KB_TRAIN` / `_EVAL`	1 / 40 / 10	0 removes the g6 tier entirely	–
`TB_GPU_INSTANCE`	g6.xlarge	`g5.xlarge` (A10G) also works, at $1.01/h vs $0.80/h	set at deploy (step 4)
`TB_SUBSTRATES` / `TB_LIMIT` / `TB_TASKS`	all	build and validate one half, or a few tasks first	–

Source: `run.sh`, `train.sh`, `5_run.sh`, `data/*.jsonl` (197 train rows); checked 5 Oct 2026.

m5 isolation and hardening

Table 39 Isolation per episode

	microVM	m5 (warm)
Per episode	own microVM, own kernel	own VM while in use; containers, networks, volumes removed by `down -v`
Shared	read-only image	with later episodes on the same VM: kernel and cached images, after a verified reset
Agent ↔ sidecars	n/a	project network only, as upstream

Table 40 Compose rewrite per episode

- fixed `container_name` → `<project>-<service>`, old name kept as a network alias
- sidecar `ports:` dropped; client publishes only `:8080` on a random port
- `./` bind mounts point into the episode's own task-dir copy
- privileged, host network or pid, devices, `docker.sock`: refused

Table 41 Host hardening

- **Role:** ECR pull on `acs-tbhdk-*` only.
- **IMDSv2**, hop limit 1, and `DOCKER-USER` drops container traffic to 169.254.169.254.
- **Containers cannot reach the host**: `INPUT` drop on `docker0`/`br-*` (agent, sshd, daemon).
- **Agent API** needs a per-instance secret and accepts only the trainer's SG.
- **Docker address pool** `10.200.0.0/16`, so it never shadows the VPC's `172.31/16`.
- **Hidden tests** are injected only at grading.
- **Verified locally:** 3 concurrent `postgres-csv-clean` episodes each got their own DB and port; nothing leaked after `down`.

Source: `m5_agent/compose_agent.py`, `m5_agent/bootstrap.sh`, `ec2_episode_sandbox.py`; `docs/m5-per-episode-isolation.md`; checked 4 Oct 2026.

Lessons: what broke on the way and how it was fixed

Table 42 Problems found while building and running the demo

Symptom	Cause	Fix
x86 task images crash at start	any `strip` breaks python-build-standalone on x86_64	use the `install_only_stripped` release asset
Upstream compose collides across episodes on one host	fixed `container_name`, host ports, `./` mounts	per-episode rewrite from `docker compose config`
Eval killed by `UnboundLocalError`	`except ... as e` shadowed the engine index	renamed (also fixed in `demo_cpu_terminalbench`)
An eval stalled	9 retries of a non-idempotent 4-min `exec`	`INVOKE_EXEC_MAX_RETRIES=1`; m5 read timeout $>$ command
Agent's bash tool hung after a timeout	detached children kept the pipe open	`session_runner` no longer waits on them
Final checkpoint save hung (smoke 2)	`expandable_segments` $\rightarrow$ `pidfd_getfd` EPERM (no `CAP_SYS_PTRACE`)	`PYTORCH_CUDA_ALLOC_CONF` made opt-in
g6 Runtime requests failed to parse	Instance Runtimes forward **chunked** bodies	`acs_http_server_chunked.py`
KernelBench image over 2 GB	CUDA torch is $\sim$2.7 GB	145 MB image, torch installed at start
All g6 episodes waited 240 s	cap 8 = one group, nothing could pre-open	cap 32, 5 batches ahead
KernelBench groups all 1s	copying `Model` to CUDA passed	speedup-gated reward
Tasks broken upstream	deleted repo, 404 downloads	`word2vec` patched; `fix-git`, `leelachess0` dropped
`train48` hung 8 h at the step-20 eval	the paused training loop kept all 24 warm m5s busy	warm pool tracks the holding loop; reclaim on supersede

Source: Commits `40581fa`, `d087d5a`, `0dc6f8e`, `3b26159`; demo README; checked 4 Oct 2026.

References

Table 43 References

Ref	Source	Supports
[Q1]	AgentCore quotas: `docs.aws.amazon.com/bedrock-agentcore/latest/devguide/bedrock-agentcore-limits.html`	1000 Runtimes/account · image $\leq$ 2 GB · 15-min request and idle timeouts
[Q2]	`aws service-quotas list-service-quotas` (us-west-2, 4 Oct 2026)	every quota code on slide 30
[D1]	AgentCore Runtime developer guide	microVMs are arm64 only; GPUs only on Instances (capacity providers)
[D2]	`docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html`	session = dedicated microVM; 15-min idle, 8 h max lifetime
[T1]	`github.com/laude-institute/terminal-bench`	the 241 tasks
[P1]	`aws.amazon.com/bedrock/agentcore/pricing`	microVM and Instance session pricing
[P2]	`aws.amazon.com/ec2/pricing/on-demand`	m5, g6e On-Demand prices
[T2]	`github.com/ScalingIntelligence/KernelBench`	the KernelBench tasks

Source: AWS documentation and service-quotas CLI, us-west-2; checked 4 Oct 2026.